IoT Gas Leakage Monitor

An Advanced Real-Time Hazardous Gas Detection & Safety Alert Ecosystem

An **IoT-Based Gas Leakage Monitoring & Alert System** is a critical safety deployment designed to continuously sample ambient air quality for combustible or toxic gas compounds (such as LPG, butane, propane, or methane). The embedded logic immediately switches on physical warning systems locally and forwards warning payloads to cloud management dashboards if an emergency occurs.

1. Project Architecture Overview

The system is built on top of a highly responsive reactive architecture to minimize data pipeline latency when a hazard materializes:

1. **Perception Layer:** The electrochemical MQ-2 gas sensor tests air particles continuously and yields an analog voltage profile that rises linearly with the environmental gas concentration.
2. **Processing Layer:** A NodeMCU (ESP8266) core ingests the input voltages via an internal Analog-to-Digital Converter (ADC). It evaluates parameters against safety lines and generates structural output switches to drive a physical auditory buzzer.
3. **Cloud Layer:** Integrated Wi-Fi modules establish continuous handshakes with cloud brokers (Blynk API), populating visual graph trends and firing push notices or remote emails automatically.

2. Bill of Materials (Components Required)

Component	Functional Purpose	Interface Architecture / Type
NodeMCU ESP8266	Cost-effective microcontroller breakout module embedded with integrated 802.11 b/g/n Wi-Fi components.	Microcontroller Core
MQ-2 Gas Sensor	Tin Dioxide (SnO_2) based air-quality sensor optimized for LPG, Propane, Hydrogen, and Smoke profiles.	Analog Output Voltage (A0 Pin)
5V Active Buzzer	High-output acoustic sounder for local near-field hazard awareness.	Digital Input/Output Line
5V Relay Module	Electromechanical isolating switch interface used to toggle heavy exhaust machinery or isolation valves.	Digital Auxiliary Control
Breadboard & Wires	Solderless infrastructure board matching standardized hookup jumper cables.	Prototyping Framework
Blynk Cloud Ecosystem	IoT application dashboard supporting rapid configuration of mobile phone notification hooks.	Cloud Interface Service

3. Circuit Connections & Wiring

The MQ-2 sensing assembly relies on an internal heating element that requires a stable 5V input rail to operate accurately, while the underlying NodeMCU handles I/O signals at a 3.3V reference. The development board accommodates this requirement via its physical `Vin` routing terminal when powered over a USB connection.

Hardware Pin Mapping:

- **MQ-2 Gas Sensor to NodeMCU:**
  - *VCC* → NodeMCU *Vin* (5V Regulated Bus Supply)
  - *GND* → NodeMCU *GND* (Common Ground Rail)
  - *AO* (Analog Output) → NodeMCU *A0* (Hardware ADC Input channel)
- **Active Buzzer to NodeMCU:**
  - *Positive (+) Input* → NodeMCU *D1* Pin (Configured as GPIO 5)
  - *Negative (-) Input* → NodeMCU *GND* Terminal

4. Software Implementation

The source file below is coded in C++ using the Arduino development environment. It pairs with the Blynk app infrastructure to manage message routing matrices.

```
#define BLYNK_TEMPLATE_ID "YOUR_TEMPLATE_ID"
#define BLYNK_TEMPLATE_NAME "Gas Leak Detector"
#define BLYNK_AUTH_TOKEN "YOUR_AUTH_TOKEN"

#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

// --- Network Settings ---
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "YOUR_WIFI_SSID";
char pass[] = "YOUR_WIFI_PASSWORD";

// --- Pin Definitions ---
const int gasPin = A0;   // MQ-2 Analog pin
const int buzzerPin = D1; // Active Buzzer pin

// --- Threshold Adjustment ---
// The ESP8266 ADC maps inputs across a 10-bit range (0 to 1023).
const int gasThreshold = 450;

BlynkTimer timer;
bool isAlertActive = false;

void checkGasLevel() {
  int rawGasValue = analogRead(gasPin);
  Serial.print("Current Gas Level Level: ");
  Serial.println(rawGasValue);

  // Synchronize data data streams to Blynk Virtual Pin V1
  Blynk.virtualWrite(V1, rawGasValue);

  // Condition Check: Evaluate real-time value against standard limits
  if (rawGasValue > gasThreshold) {
    digitalWrite(buzzerPin, HIGH); // Drive active buzzer pin HIGH

    // Evaluate flag to suppress redundant rapid notifications
    if (!isAlertActive) {
      Blynk.logEvent("gas_alert", "CRITICAL WARNING: Gas leakage detected!");
      Serial.println("ALERT SENT: Gas leakage detected!");
      isAlertActive = true;
    }
  }
  else {
    digitalWrite(buzzerPin, LOW); // Pull active buzzer pin LOW
```

```
    isAlertActive = false; // Reset warning state flag
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(buzzerPin, OUTPUT);
  digitalWrite(buzzerPin, LOW);

  // Establish cloud link configurations
  Blynk.begin(auth, ssid, pass);

  // Establish a persistent recurring execution thread timer (1500 ms)
  timer.setInterval(1500L, checkGasLevel);
}

void loop() {
  Blynk.run();
  timer.run();
}
```

5. Setting Up the Blynk Cloud & Mobile App

Follow these operational parameters to bind your physical microcontroller safely to the mobile reporting application:

1. Log into your administration workspace panel on **Blynk.cloud**.
2. Access **Datastreams** → **New Datastream** → **Virtual Pin**, assigning it explicitly to `Virtual Pin V1` (Data Type: Integer, Range: 0 to 1023).
3. Drag a **Gauge Widget** or historical **SuperChart** object into your canvas, mapping its visual link parameters to read directly from Virtual Pin V1.
4. Navigate to the **Events** manager configuration tab and declare an explicit event tag named exactly `gas_alert`. Toggle the switch option to activate "Send push notifications to mobile app".
5. Upload the complete sketch package onto the NodeMCU processor. Test the system safely by exposing the MQ-2 sensor module to unlit gas from a standard utility lighter to confirm execution of local alarms and instant push updates.

6. Advanced Scope & Upgrades

- **Solenoid Valve Interlocking Protection:** Route your controller output to a heavy-duty relay interfacing a 12V mechanical gas solenoid valve. When threshold limits break, the device isolates the incoming primary fuel feed lines immediately.

- **GSM Cellular Telemetry Back-up:** In locations lacking consistent Wi-Fi stability, integrate a **SIM800L GSM Breakout** module into the system bus. This addition ensures immediate fallback SMS transmissions or telephone alerts occur even during local internet disruptions.
- **Calibration & PPM Conversion Profiles:** Integrate mathematical algorithms using inverse sensor load equations derived from the MQ-2 log-log sheets. This converts raw ADC integers into accurate **PPM (Parts Per Million)** indicators.